Recurrent Neural Networks

- Motivation:
  - Neural network model, but with a state
  - How can we borrow ideas from FSAs?

- RNNs are FSAs …
  - With a twist
  - No longer finite in the same sense
  - The state is an embedding of the history in a continuous space
  - The embedding function of the history to a vector is learned as well

Recurrent Neural Networks

- Map from dense sequence to dense representation
  - Maps a sequence of vector inputs to a sequence of vector states

$$x_1, ..., x_n \rightarrow s_1, ..., s_n$$

  - A (parametrized) transition function *R* does the mapping:

$$s_i = R(s_{i-1}, x_i)$$

  - *R* is parametrized and parameters are shared across steps

$$s_4 = R(s_3, x_4) = R(R(s_2, x_3), x_4) = R(R(R(s_1, x_2), x_3), x_4)$$

Recurrent Neural Networks

- An output function *O* maps states to (vector) outputs, which are often viewed as a distribution over the possible labels

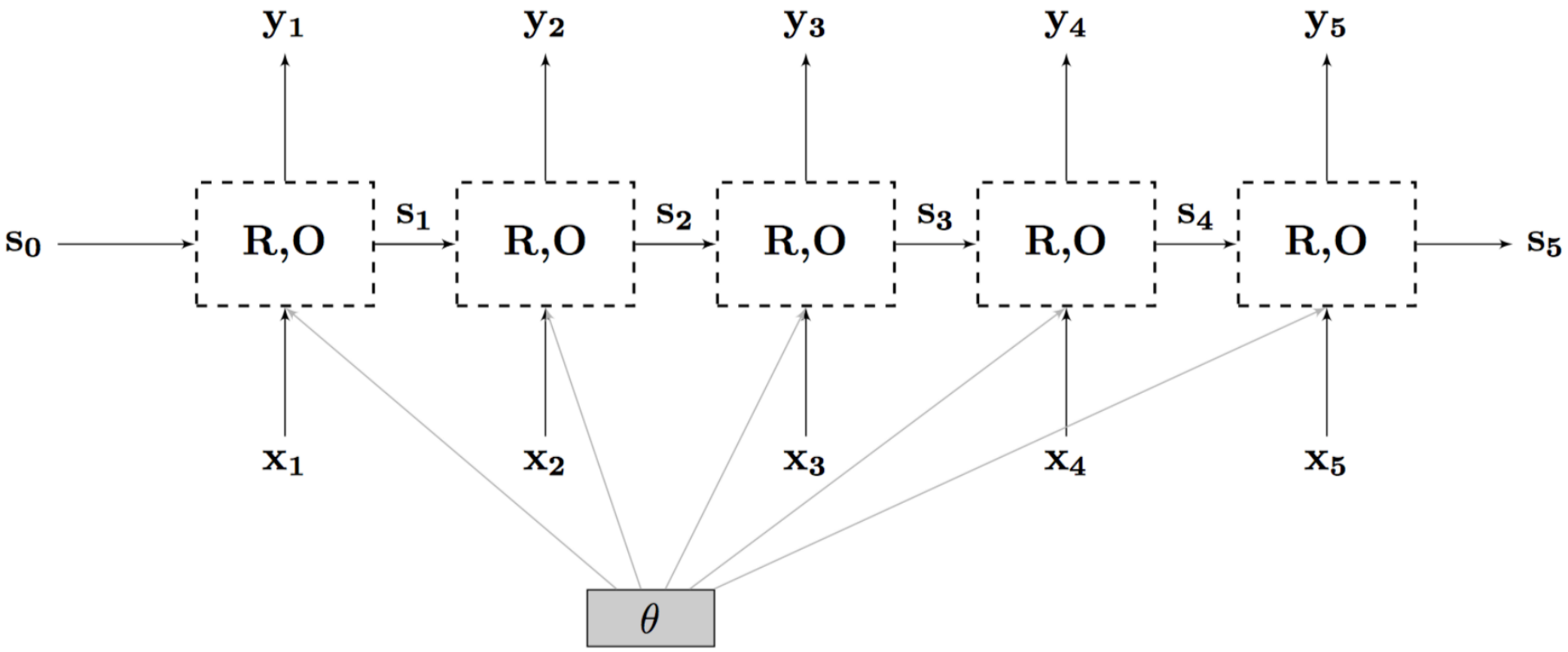
- Example:

$$R_{Elman}(s, x) = tanh(W[x, s] + b)$$

$$O(s_i) = softmax(W's_i + b')$$

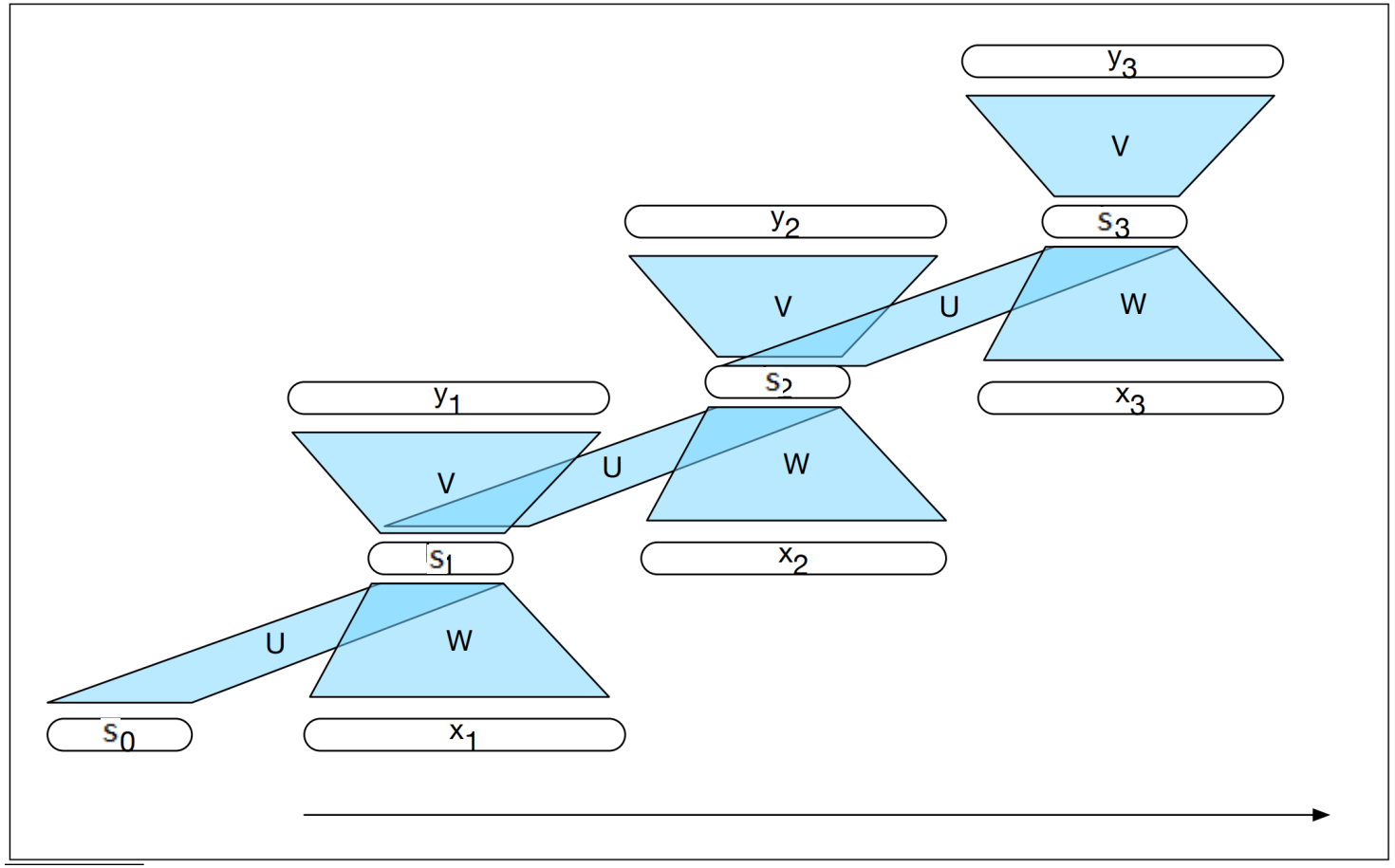
- Conceptually:
  - *R* is the function embedding the history in a vector
  - *O* is the function relating the embedded history and the output
  - They are learned jointly

RNNs: Graphical Representation



Recurrent Neural Language Models

- An “unrolled” view of an RNN
- The network stays the same at each time stamp. What’s different are the inputs and previous hidden states



Reminder: Loss Functions

- Training is done by defining a loss function (i.e. a function that determines how "bad" a classification is relative to the gold standard)
- The network then attempts to minimize the empirical risk. For a sample $S=\{x_1,\ldots,x_m\}$ with gold standard labels $\{y_1,\ldots,y_m\}$, the empirical risk is:

$$L_S(\theta) = \frac{1}{m}\sum_{i=1}^{m} \ell(\theta; (x_i, y_i))$$